MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 1

DIGITAL LOGIC SYSTEM

SECTION 1

GROUP 5

SUBMISSION DATE: 12 OCTOBER 2026

NO.	MATRIC NO.	NAME
1	2417497	MUHAMMAD AIDIL HAZIM BIN SHAHARUL AMIR
2	2413393	MUHAMMAD DANISH BIN ZAINUDDIN
3	2415821	MUHAMMAD DANISH HAIQAL BIN MOHD EFENDY
4	2417169	MUHAMMAD DANISH NAJWAN BIN MOHD ALI

Table of Contents

SEGMENT DISPLAY INTERFACING....3
1.0 Introduction & Theory....3
1.1 Materials & Equipment....3
1.2 Experimental Setup....3
1.3 Methodology & Programming Logic....4
1.4 Data Collection & Analysis....7
1.5 Results & Discussion....8
RECOMMENDATIONS....10
ACKNOWLEDGEMENT....10
STUDENT DECLARATION7-....11

ABSTRACT

This experiment investigated implementing a mechatronic digital display system using direct General Purpose Input/Output (GPIO) pin control. The primary objective was to interface a common-cathode seven-segment display with an Arduino Uno microcontroller to demonstrate fundamental digital logic operations, signal handling, and hardware-software synchronization. Seven-segment pins (A–G) were connected through 220 Ω current-limiting resistors to individual digital outputs, while pushbuttons with 10 kΩ pull-up resistors provided manual increment and reset control signals. Custom embedded C++ software evaluated input pin states and output binary logic levels (HIGH/LOW) to drive the corresponding segments. Experimental validation confirmed that the display accurately cycled through numeric digits 0 to 9 in response to manual button triggers without signal ambiguity. Overall, the direct GPIO control method provided a simple and reliable approach for visual feedback in embedded applications, establishing a foundational baseline for complex display interfacing and digital circuit design.

7-SEGMENT DISPLAY INTERFACING

1.0 Introduction & Theory

The purpose of this experiment is to explore the fundamentals of digital logic systems, basic logic gates, and electronic circuit interfacing. The core focus of this session is practical IC-based interfacing applications, specifically controlling a 7-segment display through a microcontroller. The expected outcome is a fully functional circuit where a user can manually control the sequential numbering of the display using physical input buttons.

1.1 Materials & Equipment

- Arduino Uno board
- 7-segment LED display
- Resistors , 10k, 220
- Pushbuttons
- Breadboard & Jumper Wires
- PC with Python Environment

1.2 Experimental Setup

1. The circuit was assembled on a breadboard connecting the microcontroller to the display and pushbuttons according to the following specific pin assignments:
    - 7 Segment Display Interfacing: Each segment pin (a through g) was connected to separate digital output pins on the Arduino Uno. The common cathode pin was connected to the Arduino's 5V pin.

Component Pin	Segment A	Segment B	Segment C	Segment D	Segment E	Segment F	Segment G
Arduino Uno Pin	3	2	8	7	6	4	5

    - Pushbutton Interfacing: The increment pushbutton was connected to Digital Pin 11, and the reset pushbutton was connected to Digital Pin 10, with their opposite terminals tied to GND. Internal pull-up resistors were enabled via software

2. Hardware configuration instructions.

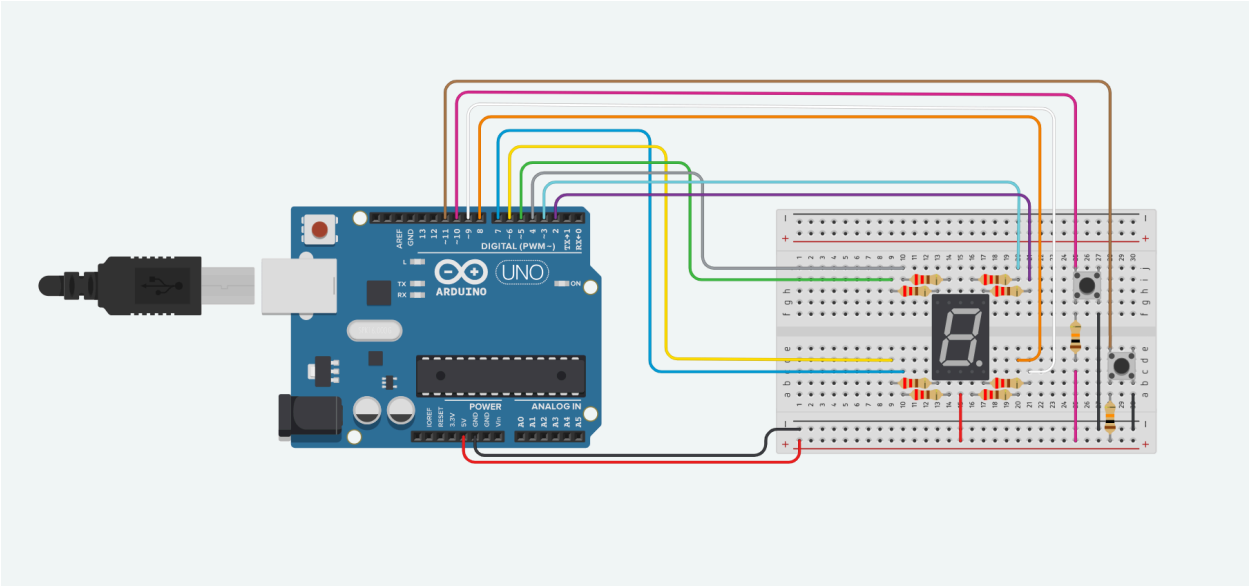


Figure 1.1: Circuit Assembly Diagram

1.3 Methodology & Programming Logic

The experiment steps :

1. Build the circuit according to the circuit setup instructions.
2. Upload the provided Arduino code to our Arduino Uno.
3. Open the Serial Monitor in the Arduino IDE.
4. Press the increment button to increase the count. The 7-segment display should show the numbers from 0 to 9 sequentially.
5. Press the reset button to reset the count to 0.

Coding :

```
// Define the pins for each segment (D0 to D6)
const int segmentA = 3;
const int segmentB = 2;
const int segmentC = 8;
const int segmentD = 7;
const int segmentE = 6;
const int segmentF = 4;
const int segmentG = 5;

// Define the pins for the push buttons
const int buttonIncrementPin = 11;
const int buttonResetPin = 10;

// Variables to track display state
int currentDigit = 0;

// Debouncing variables for Increment Button
int lastIncState = HIGH;
int incButtonState = HIGH;
unsigned long lastIncDebounceTime = 0;

// Debouncing variables for Reset Button
int lastResetState = HIGH;
int resetButtonState = HIGH;
unsigned long lastResetDebounceTime = 0;

const unsigned long debounceDelay = 50; // Debounce delay in milliseconds
```

```
// Function to update segments based on active-low logic (LOW = ON, HIGH = OFF)
void displayDigit(int digit) {
  switch (digit) {
    case 0:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, HIGH);
      break;
    case 1:
      digitalWrite(segmentA, HIGH);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
    case 2:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, HIGH);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, LOW);
      break;
```

```
case 3:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, LOW);
  break;
case 4:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
case 5:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
case 6:
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
  break;
```

```
  case 7:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, HIGH);
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, HIGH);
    digitalWrite(segmentG, HIGH);
    break;
  case 8:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, LOW);
    digitalWrite(segmentE, LOW);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, LOW);
    break;
  case 9:
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, LOW);
    digitalWrite(segmentE, HIGH);
    digitalWrite(segmentF, LOW);
    digitalWrite(segmentG, LOW);
    break;
```

```
void setup() {
  Serial.begin(9600);

  // Initialize segment pins as OUTPUTs
  pinMode(segmentA, OUTPUT);
  pinMode(segmentB, OUTPUT);
  pinMode(segmentC, OUTPUT);
  pinMode(segmentD, OUTPUT);
  pinMode(segmentE, OUTPUT);
  pinMode(segmentF, OUTPUT);
  pinMode(segmentG, OUTPUT);

  // Initialize both button pins with internal pull-up resistors
  pinMode(buttonIncrementPin, INPUT_PULLUP);
  pinMode(buttonResetPin, INPUT_PULLUP);

  // Display initial digit 0
  displayDigit(currentDigit);
  Serial.println("7-Segment Counter with Reset Ready!");
  Serial.print("Displaying Digit: ");
  Serial.println(currentDigit);
}
```

```
void loop() {
  // Read current states of both buttons
  int readingInc = digitalRead(buttonIncrementPin);
  int readingReset = digitalRead(buttonResetPin);

  // --- Increment Button Logic ---
  if (readingInc != lastIncState) {
    lastIncDebounceTime = millis();
  }
  if ((millis() - lastIncDebounceTime) > debounceDelay) {
    if (readingInc != incButtonState) {
      incButtonState = readingInc;
      // Trigger on button press (transition to LOW)
      if (incButtonState == LOW) {
        currentDigit = (currentDigit + 1) % 10; // Increment and cycle 0-9
        displayDigit(currentDigit);
        Serial.print("Displaying Digit: ");
        Serial.println(currentDigit);
      }
    }
  }
  lastIncState = readingInc;
```

```
// --- Reset Button Logic ---
if (readingReset != lastResetState) {
  lastResetDebounceTime = millis();
}
if ((millis() - lastResetDebounceTime) > debounceDelay) {
  if (readingReset != resetButtonState) {
    resetButtonState = readingReset;
    // Trigger on button press (transition to LOW)
    if (resetButtonState == LOW) {
      currentDigit = 0; // Reset count to 0
      displayDigit(currentDigit);
      Serial.println("Counter Reset to 0!");
    }
  }
}
lastResetState = readingReset;
```

1.4 Data Collection & Analysis

The results were clearly observed by us.

Bil	Input	Output
1	The Arduino is turned on	The 7-segment displays ‘0’
2	The pushbutton_1 is pressed	The 7-segment display value increments to ‘1’
3	The pushbutton_1 is pressed	The 7-segment display value increments to ‘2’
4	The pushbutton_1 is pressed	The 7-segment display value increments to ‘3’
5	The pushbutton_1 is pressed	The 7-segment display value increments to ‘4’
6	The pushbutton_1 is pressed	The 7-segment display value increments to ‘5’

7	The pushbutton_1 is pressed	The 7-segment display value increments to ‘6’
8	The pushbutton_1 is pressed	The 7-segment display value increments to ‘7’
9	The pushbutton_1 is pressed	The 7-segment display value increments to ‘8’
10	The pushbutton_1 is pressed	The 7-segment display value increments to ‘9’
11	The pushbutton_1 is pressed	The 7-segment display value increments to ‘0’
12	The pushbutton_2 is pressed	The 7-segment displays ‘0’
13	The pushbutton_2 is pressed when the 7-segment displays ‘4’	The 7-segment displays ‘0’

The data confirmed that:

1. The pushbutton_1 is set to the increment function.
2. The pushbutton_2 is set to reset the current value to zero.
3. The display followed the correct segment configuration defined in the code.

Digit	**Segment**	**Display**
0	a,b,c,d,e,f	Correct
1	b,c	Correct
2	a,b,d,e,g	Correct
3	a,b,c,d,g	Correct
4	b,c,f,g	Correct
5	a,c,d,f,g	Correct
6	a,c,d,e,f,g	Correct
7	a,b,c	Correct
8	a,b,c,d,e,f,g	Correct
9	a,b,c,d,f,g	Correct

1.5 Results & Discussion

The result of the experiment successfully shows manual control of the 7-segment display using Arduino Uno and push buttons :

- The 7-segment displays the correct value of the current value stored in the Arduino Uno.
- The value increases by one when the pushbutton_1 is pressed.
- The value then successfully resets to zero when the second push button is pressed.
- The segments of the display are correctly connected to the Arduino code logic.
- The display worked correctly without error or flickering.

The discussion :

1. Interpretation of Results and Implications

- The output achieved from the 7-segment display shows that the circuit is well connected to the Arduino Uno. It also shows that the code used is suitable for the experiment. In addition, the push buttons used to increment and reset the value also show integration of input and output, showing the result as intended.

2. Discrepancies Between Expected and Observed Outcomes

- There were no major discrepancies observed during the experiment and everything went as intended.

3. Sources of Error and Improvement

- Mechanical Button Bounce : Push buttons naturally produce multiple electrical signals once pushed, producing multiple signals. By using a debouncer hardware, this situation can be avoided.

- Human Reaction Time: Since the reset and increment push buttons are manually pressed, the pressing speed and duration may vary, resulting in inconsistent results or missed counts.

Question: How can you interface an I2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED

An I2C LCD is interfaced using only two pins (SDA and SCL) through the I2C protocol, allowing Arduino to send data using simple commands with libraries like LiquidCrystal_I2C. This makes the coding easier and reduces wiring. In contrast, a 7-segment display requires multiple digital pins to control each segment individually, and numbers are formed by turning segments ON or OFF manually. An LED matrix uses row-column scanning (multiplexing), which requires more complex coding and

timing control to display patterns. Overall, the I2C LCD is the simplest and most efficient, while the 7-segment and LED matrix require more control and complexity.

1.6 Conclusion

This project successfully demonstrated the design and implementation of a seven-segment display system using an Arduino microcontroller. The system was able to display numerical digits from 0 to 9 accurately by controlling the output signals sent to each segment. After going through the process of putting the circuit together, coding it, and testing, the project achieved its main objective that is to understand and implement digital logic interfacing between Arduino and external display devices, such as a 7-segment display.

In terms of understanding, we now know how to code a counting machine, and as for implementing, we are confident in the reenactability of this project. The results proved that even simple electronic systems can be effectively developed through systematic design and logical programming.

For an Islamic reflection, this project highlights the importance of seeking knowledge (Ilm') seen in our objective to understand its inner workings. Knowledge is a gift from Allah that enables mankind to understand and control the world around them. By exploring the principles of electronics and control systems, we recognize the precision and order that Allah has placed in creation.

RECOMMENDATIONS

1. Replace direct GPIO wiring with a serial-in/parallel-out shift register (e.g., 74HC595) or an I2C I/O expander because driving a 7-segment display using seven or more dedicated digital pins only to show a single numeric digit is highly inefficient.
2. Upgrade the hardware platform to an IoT-enabled microcontroller such as an ESP32 to enable remote monitoring rather than being restricted to a localized physical setup.

REFERENCES

Arduino. (2026). *UNO R3* [Hardware documentation]. Arduino Docs. https://docs.arduino.cc/hardware/uno-rev3/

Oasistek. (n.d.). *TOS-5121 0.52-inch single-digit 7-segment LED display datasheet*. https://www.alldatasheet.com/datasheet-pdf/pdf/89363/ETC/TOS5121.html

Prabhu, A. (2026, January 20). *How seven segment display works & interface it with Arduino*. Last Minute Engineers. https://lastminuteengineers.com/seven-segment-arduino-tutorial/

APPENDICES

Video link to our group work :

- https://vt.tiktok.com/ZSb7fGtmv/
- https://vt.tiktok.com/ZSb7f7Jgx/

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the report by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the report is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: Muhammad Aidil Hazim bin Shaharul Amir Matric No: 2417497	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish bin Zainuddin Matric No: 2413393	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish Haiqal bin Mohd Efendy Matric No: 2415821	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish Najwan bin Mohd Ali Matric No: 2417169	Read [✔] Understand [✔] Agree [✔]